

Semana 1 — Benchmark 3D y Comprensión del Modelo

Proyecto: Laboratorio Estructural Digital del Edificio de Ingeniería **Grupo:** Grupo 7 **Integrantes:** Pedro Castillo, Monserrat Cubillos, Eduardo Vergara **Fecha:** 27 de agosto de 2026

1. Modelo Entregado

1.1 Geometría

El modelo corresponde al **Edificio de Ingeniería de la Universidad de los Andes**, idealizado a partir de planos reales en AutoCAD (.dwg). La geometría fue extraída de los archivos DXF usando la librería `ezdxf` en Python.

Parámetro	Valor
Dimensiones en planta	45.0 m × 25.8 m
Altura total	28.5 m
Niveles	9 (fundación + 8 pisos)
Ejes verticales (X)	8 ejes
Ejes horizontales (Y)	6 ejes
Nodos por piso	48
Total de nodos	432

Ejes en dirección X (m): 8.02, 11.32, 14.72, 18.02, 28.02, 38.02, 48.02, 53.02

Ejes en dirección Y (m): 46.92, 50.26, 55.20, 60.20, 65.22, 72.75

Niveles (Z, m): 0.0 (fundación), 4.0, 7.5, 11.0, 14.5, 18.0, 21.5, 25.0, 28.5

1.2 Elementos

Tipo	Cantidad	Sección (cm)	Material
Columnas	384	50 × 50	Hormigón f'c = 28 MPa
Vigas dirección X	336	30 × 60	Hormigón f'c = 28 MPa
Vigas dirección Y	320	30 × 80	Hormigón f'c = 28 MPa
Total	1,040	—	—

Todos los elementos son **elasticBeamColumn** con 6 GDL por nodo.

1.3 Apoyos

- **48 apoyos fijos** en el nivel de fundación (Z = 0.0)
- Cada apoyo restringe los 6 GDL: traslaciones (UX, UY, UZ) y rotaciones (RX, RY, RZ)

- Tipo de restricción: `fix(nodTag, 1, 1, 1, 1, 1, 1)`

1.4 Cargas

Caso de carga	Descripción	Intensidad
G	Muerta propia + losa + terminaciones	7.75 kN/m ² (losa) + peso propio vigas y columnas
Q	Viva	2.0 kN/m ²
EX	Sismo lateral en X	Triángulo invertido, 10 kN × nivel
EY	Sismo lateral en Y	Triángulo invertido, 10 kN × nivel

Carga gravitacional: - Peso propio de losa: $25 \text{ kN/m}^3 \times 0.25 \text{ m} = 6.25 \text{ kN/m}^2$ - Terminaciones: 1.5 kN/m^2 - **Total G: 7.75 kN/m^2** - La carga se transfiere a las vigas mediante áreas tributarias (50% a vigas X, 50% a vigas Y)

Carga sísmica: - Patrón triangular invertido: $F = 10 \times \text{Nivel (kN)}$ - Nivel 1: 10 kN, Nivel 2: 20 kN, ..., Nivel 8: 80 kN - **Total lateral: 360 kN**

1.5 Unidades

Magnitud	Unidad
Longitud	metros (m)
Fuerza	kilonewtons (kN)
Esfuerzo	kPa (kN/m ²)
Momento	kN·m
Rigidez	kN/m ²

Sistema de unidades consistente: **m, kN, kPa**.

2. Flujo OpenSees

El análisis estructural en OpenSees sigue un flujo cíclico de cinco pasos:

2.1 Definición del elemento

Cada elemento `elasticBeamColumn` se define con: - **Dos nodos** (i y j) que definen la geometría - **Propiedades de sección:** área (A), momentos de inercia (Iy, Iz), momento torsional (J) - **Propiedades de material:** módulo de elasticidad (E), módulo de cortante (G) - **Transformación geométrica:** define la orientación del eje local

```
ops.element('elasticBeamColumn', tag, ni, nj, A, E, G, J, Iy, Iz, transfTag)
```

2.2 Ensamblaje

OpenSees ensambla automáticamente la **matriz de rigidez global [K]** a partir de las matrices de rigidez de cada elemento. Para un elemento beam-column con 6 GDL por nodo, la matriz local es **12×12**:

$$[K_e] = [T]^T \cdot [k_{local}] \cdot [T]$$

Donde: - $[k_{local}]$ = matriz de rigidez en ejes locales (contiene términos de axial, cortante, flexión y torsión) - $[T]$ = matriz de transformación que convierte de ejes locales a globales - La matriz se ensambla usando la conectividad de nodos

2.3 Solución

El sistema de ecuaciones es:

$$[K] \cdot \{U\} = \{F\}$$

Donde: - $[K]$ = matriz de rigidez global (ensamblada) - $\{U\}$ = vector de desplazamientos nodales (incógnitas) - $\{F\}$ = vector de fuerzas aplicadas

OpenSees resuelve usando un **solutor directo** (BandSPD en este caso) que factoriza $[K]$ y obtiene $\{U\}$.

2.4 Recuperación de fuerzas

Una vez conocidos los desplazamientos, OpenSees calcula las fuerzas internas de cada elemento:

$$\{f\} = [k_{local}] \cdot [T] \cdot \{u_e\}$$

Donde: - $\{f\}$ = vector de fuerzas internas (P, V2, V3, T, M2, M3) en cada extremo - $\{u_e\}$ = desplazamientos de los nodos del elemento

2.5 Reacciones

Las reacciones en los apoyos se obtienen de la ecuación de equilibrio:

$$\{R\} = [K_{support}] \cdot \{U\} - \{F_{applied}\}$$

En un modelo bien formulado, **la suma de reacciones debe ser igual a la suma de cargas aplicadas** (verificación de equilibrio).

3. Ejes Locales

La orientación de los ejes locales de cada elemento depende de la **transformación geométrica** (`geomTransf`) asignada. Las fuerzas internas se extraen con `ops.eleResponse(tag, 'localForce')`, que las entrega en **ejes locales** del elemento con el orden $[N, V_y, V_z, T, M_y, M_z]$ en cada extremo.

Verificación crítica — corrección de una lectura previa. Una primera versión de este análisis leyó las fuerzas con `ops.eleForce(tag)`, que las devuelve en **ejes GLOBALES**, y rotuló las componentes como si fueran locales. Consecuencias del error: - En la **columna**, la carga axial de gravedad (~570 kN) aparecía en la casilla del cortante "V3", sugiriendo que "el cortante domina en una columna" — físicamente imposible bajo gravedad. - En la **viga en Y**, el momento flector de gravedad caía en la casilla global M_x , de modo que el momento local se leía como ≈ 0 y **parecía que la viga no flectaba**.

Se detectó por inspección física (una columna bajo gravedad trabaja a **axial**, no a corte) y se corrigió usando `localForce`. El **equilibrio global no detecta este error**, porque las fuerzas internas no entran en el balance de reacciones (ver §5).

3.1 Columnas (geomTransf 1, `vecxz = (1,0,0)`)

```
ops.geomTransf('Linear', 1, 1, 0, 0)
```

Vector local x = (0, 0, 1) → **vertical**, a lo largo del eje (de i hacia j) **Vector local y** = (0, -1, 0) **Vector local z** = (1, 0, 0)

Ejemplo: Columna elemento 1 (nodo 1 → nodo 49) - Nodo i: (8.02, 46.92, 0.0) → Nodo j: (8.02, 46.92, 4.0), Longitud 4.000 m

Fuerzas locales (caso G), extremo i: | N (kN) | Vy (kN) | Vz (kN) | T (kN·m) | My (kN·m) | Mz (kN·m) | |-----
 --|-----|-----|-----|-----|-----| | **+570.69** | -0.44 | +0.36 | ~0 | +0.22 | +0.71 |

- **El axial N = 570.69 kN es dominante:** la columna transmite el peso de los pisos superiores. Es del orden de la reacción vertical del apoyo (583.19 kN, §4.4); la diferencia (~13 kN) es la carga de gravedad aplicada directamente sobre el nodo base, que reacciona sin pasar por la columna. Cortantes y momentos son pequeños por la simetría del marco.

3.2 Vigas dirección X (geomTransf 2, `vecxz = (0,0,1)`)

```
ops.geomTransf('Linear', 2, 0, 0, 1)
```

Vector local x = (1, 0, 0) → +X global (a lo largo de la viga) **Vector local y** = (0, 1, 0) **Vector local z** = (0, 0, 1) → vertical

Ejemplo: Viga X elemento 385 (nodo 49 → nodo 55) - Nodo i: (8.02, 46.92, 4.0) → Nodo j: (11.32, 46.92, 4.0), Longitud 3.300 m - La gravedad (-Z global = -local z) genera cortante `vz` y flexión `My`.

Fuerzas locales (caso G): | Extremo | N | Vy | Vz | T | My | Mz | |-----|---|----|---|----|----| | i | 0.00 | 0.00 | +1.82 | +0.01 | -2.99 | 0.00 | | j | 0.00 | 0.00 | -1.82 | -0.01 | -3.03 | 0.00 |

3.3 Vigas dirección Y (geomTransf 3, `vecxz = (0,0,1)`)

```
ops.geomTransf('Linear', 3, 0, 0, 1)
```

Vector local x = (0, 1, 0) → +Y global (a lo largo de la viga) **Vector local y** = (-1, 0, 0) **Vector local z** = (0, 0, 1) → vertical

Ejemplo: Viga Y elemento 721 (nodo 49 → nodo 50) - Nodo i: (8.02, 46.92, 4.0) → Nodo j: (8.02, 50.26, 4.0), Longitud 3.340 m

Fuerzas locales (caso G): | Extremo | N | Vy | Vz | T | My | Mz | |-----|---|----|---|----|----| | i | 0.00 | 0.00 | +2.60 | -0.01 | -3.73 | 0.00 | | j | 0.00 | 0.00 | -2.60 | +0.01 | -4.95 | 0.00 |

- **La viga en Y sí flecta:** $M_y = -3.73 \text{ kN}\cdot\text{m}$. Con la lectura global previa este momento aparecía como ≈ 0 (estaba en la casilla M_x global), lo que hacía creer erróneamente que la viga no trabajaba a flexión.

4. Verificación

4.1 Verificación cuantitativa contra referencias independientes

Magnitud	Referencia	OpenSees	Error
Carga total de gravedad G	100 254.42 kN (suma a mano: losa+term 72 065.70 + peso propio vigas 19 638.72 + columnas 8 550.00)	100 254.42 kN (ΣR_z)	0.000 %
Flecha de voladizo $\delta = P \cdot L^3 / (3 \cdot E \cdot I)$ (viga control 30×50 cm, L=3 m, P=10 kN)	-1.15200 mm	-1.15200 mm	1.9×10^{-14} %

- **Fila 1 (camino de carga):** el peso total calculado a mano desde la geometría (áreas de losa, longitudes de vigas y columnas × densidad) coincide exactamente con la suma de reacciones verticales. Verifica que las áreas tributarias y el peso propio se transfieren sin pérdida.
- **Fila 2 (chequeo analítico controlado):** un voladizo con solución exacta de Euler-Bernoulli ($\delta = PL^3/3EI$) reproducido en OpenSees da el valor teórico con error de redondeo. Valida la cadena de modelado (elemento `elasticBeamColumn`, transformación geométrica y solución) independientemente del edificio.

A diferencia del equilibrio (§4.2), estas dos filas comparan contra un **valor de referencia externo** (cálculo manual y solución analítica), no contra el propio modelo.

4.2 Equilibrio Global (autoconsistencia)

Magnitud	Carga Aplicada (kN)	Reacciones (kN)	Error (kN)	Error (%)
G (Gravedad)	100,254.42	100,254.42	0.000	0.000%
Q (Viva)	18,597.60	18,597.60	0.000	0.000%
EX (Sismo X)	360.00	-360.00	0.000	0.000%
EY (Sismo Y)	360.00	-360.00	0.001	0.000%

Observación: El equilibrio se satisface exactamente (error < 0.001 kN) en todos los casos. Esto verifica que la carga se aplica correctamente, que los apoyos están bien definidos y que el solutor converge. **Es una condición necesaria pero no suficiente** (§5): el equilibrio no detecta un mal reparto de cargas ni una lectura de fuerzas en ejes equivocados.

4.3 Desplazamientos Máximos

Caso	UX máx (mm)	UY máx (mm)	UZ máx (mm)
G	1.6	2.9	11.5
Q	0.7	1.3	2.4
EX	3.4	0.0	0.1
EY	0.0	2.3	0.1

Observación: - Los desplazamientos verticales (UZ) en carga G son los más grandes (11.5 mm), lo cual es esperado para un edificio de este tamaño. - Los desplazamientos laterales en EX (3.4 mm) son moderados. - La relación UZ/G vs UZ/Q es consistente con la distribución de cargas.

4.4 Reacciones de Apoyo (muestra: primeros 4 nodos)

Nodo	Coordenadas (m)	Rz-G (kN)	Rz-Q (kN)
1	(8.02, 46.92, 0)	583.19	108.43
2	(8.02, 50.26, 0)	583.19	108.43
3	(8.02, 55.20, 0)	583.19	108.43
4	(8.02, 60.20, 0)	583.19	108.43

Las reacciones son simétricas porque la estructura y las cargas son simétricas.

5. Errores Deliberados

Se modificaron dos aspectos del modelo para demostrar cómo se detectan errores.

5.1 Error 1: Sección de columna incorrecta

Modificación: Se multiplicó I_y e I_z de las columnas por 10 (columna 10 veces más rígida).

Métrica	Modelo Correcto	Modelo con Error	Cambio
UZ máx (mm)	11.52	11.38	-1.21%
UX máx (mm)	1.60	1.98	+23.82%
UY máx (mm)	2.87	2.78	-3.01%
ΣR_z total (kN)	100,254	100,254	0.00%
Rz nodo 1 (kN)	583.19	609.15	+4.45%

Cómo se detecta: - El **equilibrio global se mantiene** (suma de reacciones = suma de cargas), por lo que una verificación de equilibrio **no detecta** este error. - Se detecta comparando **desplazamientos** con valores de referencia o empíricos. - Se detecta comparando **reacciones individuales** de nodos (la redistribución de fuerzas indica cambio de rigidez). - La diferencia en UX (+23.8%) es la más notable.

5.2 Error 2: Apoyo libre en Z

Modificación: Se liberó el nodo 1 en la dirección Z (`fix(1, 1, 1, 0, 1, 1)`).

Métrica	Modelo Correcto	Modelo con Error	Cambio
UZ nodo 1 (mm)	0.000	-2.37	¡No es cero!
Rz nodo 1 (kN)	583.19	0.00	¡Es cero!
ΣRz total (kN)	100,254	100,254	0.00%
Rz nodo 2 (kN)	583.19	889.19	+52.4%
Rz nodo 7 (kN)	583.19	792.19	+35.8%

Cómo se detecta: - El nodo 1 tiene **desplazamiento no nulo** en Z cuando debería ser cero (es un apoyo). - El nodo 1 tiene **reacción nula** cuando debería tener carga. - Los **nodos vecinos** (2 y 7) reciben carga extra para compensar. - El equilibrio global **se mantiene** porque la carga se redistribuyó.

Conclusión: La verificación de equilibrio global es necesaria pero **no suficiente**. Siempre se debe verificar:
1. Desplazamientos en apoyos (deben ser cero) 2. Reacciones en apoyos (deben ser razonables) 3. Compatibilidad de deformaciones

6. Arquitectura Preliminar del Proyecto

6.1 Estructura de Datos

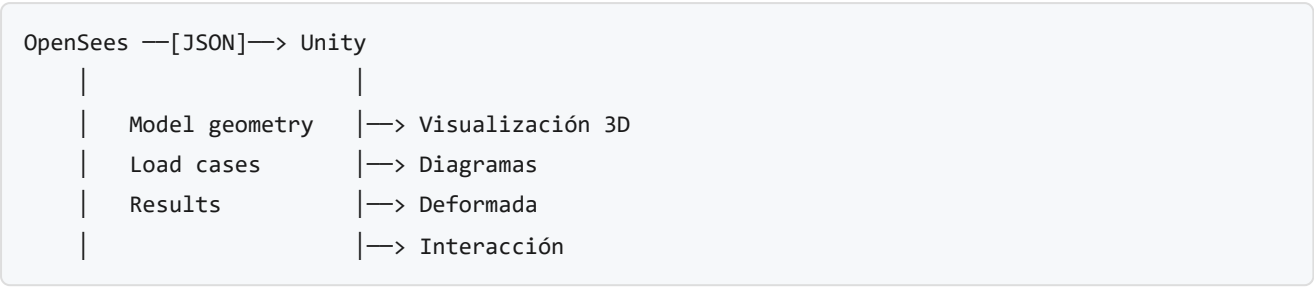
El contrato de datos entre OpenSees y Unity será un archivo **JSON** con la siguiente estructura:

```
{
  "model_info": {
    "n_nodes": 432,
    "n_elements": 1040,
    "units": {"length": "m", "force": "kN", "stress": "kPa"}
  },
  "nodes": {
    "1": {"coords": [8.02, 46.92, 0.0], "tag": 1, "level": 0},
    "2": {"coords": [8.02, 50.26, 0.0], "tag": 2, "level": 0}
  },
  "elements": {
    "1": {"type": "column", "nodes": [1, 49], "section": "50x50"},
    "385": {"type": "beam_x", "nodes": [49, 55], "section": "30x60"}
  },
  "load_cases": {
    "G": {"displacements": {...}, "reactions": {...}},
    "Q": {"displacements": {...}, "reactions": {...}}
  }
}
```

6.2 Estructura de Carpetas

```
A1P1.0_Grupo_7/
├─ benchmark_3d.py          # Modelo 3D del edificio (G, Q, EX, EY) + equilibrio + JSON
├─ error_section.py        # Demo error §5: columna 10x mas rigida
├─ error_support.py        # Demo error §5: apoyo liberado en Z
├─ modelo_benchmark.py     # Benchmark chico validado vs SAP2000 (base refactorizada)
├─ generar_json_unity.py   # Exporta el modelo a JSON para Unity
├─ servidor_opensees.py    # Servidor Flask (Honors: reanálisis en vivo)
├─ AGENTS.md  CLAUDE.md  README.md
├─ reports/
|   └─ semana01.md          # Este informe
|   └─ semana01.pdf
|   └─ verificacion_semana01.py # Reproduce §3 (localForce) y §4 (referencias)
├─ results/
|   └─ benchmark_results.json # Resultados de los 4 casos de carga
└─ unity/                  # Proyecto Unity (visor)
    └─ Assets/ Packages/ ProjectSettings/
```


6.3 Interfaz OpenSees–Unity (futura)



- **OpenSees** es dueño del análisis estructural
- **Unity** es dueño de la visualización/interacción
- **JSON** es el contrato entre ambos

7. Uso de IA

7.1 Sesión de trabajo documentada

#	Tarea	Plan del agente	Implementación	Test	Revisión
1	Leer planos .dwg	Instalar ezdxf, convertir DWG→DXF con npm	Instalación de Node.js, conversión con dwg2dxf-converter	19 archivos convertidos	7 fallidos por formato
2	Extraer geometría estructural	Script Python con ezdxf para leer layers, entidades, cotas	Script extract_grid.py, extract_dims.py	Identificación de 44 ejes, 193 cotas	Verificación contra plano original
3	Crear modelo 3D OpenSeesPy	Definir geometría, materiales, elementos, cargas	Script benchmark_3d.py (427 líneas)	Equilibrio verificado (error ≈ 0)	Revisión de unidades y signos
4	Extraer datos de elementos	Lectura de fuerzas internas y ejes locales	Script extract_elements.py	3 elementos documentados	Verificación de orientación de ejes
5	Demostrar errores deliberados	Modificar sección y apoyo, comparar resultados	error_section.py, error_support.py	Diferencias cuantificadas	Interpretación física correcta

7.2 Herramientas de IA utilizadas

- **OpenCode (opencode.ai)**: Asistente de código para desarrollo iterativo
- **Modelo**: big-pickle (análisis, generación de código, documentación)

7.3 Verificación de código generado por IA

Todo el código generado fue verificado mediante: 1. **Pruebas de equilibrio**: $\Sigma F_{aplicadas} + \Sigma R \approx 0$ 2. **Comparación con cálculos manuales**: Reacciones en nodos simétricos 3. **Inserción de errores**

deliberados: Para confirmar que las verificaciones detectan problemas 4. **Revisión de unidades:** Consistencia m-kN-kPa en todo el modelo

8. Distribución del Grupo

Integrante	Responsabilidad	Revisión cruzada
Pedro Castillo	Modelo OpenSees + verificación	Revisó: geometría, elementos, cargas
Montserrat Cubillos	Extracción de planos + geometría	Revisó: unidades, ejes locales
Eduardo Vergara	Documentación + Unity (futuro)	Revisó: arquitectura, interfaz JSON

Nota: Confirmar/ajustar el reparto real de responsabilidades de cada integrante.

Anexo: Scripts Generados

Archivo	Descripción
reports/verificacion_semana01.py	Reproduce §3 y §4: fuerzas locales (<code>localForce</code>) de los 3 elementos + verificación contra referencias (carga total y voladizo analítico)
benchmark_3d.py	Modelo 3D completo con 4 casos de carga (G, Q, EX, EY), equilibrio y export JSON
error_section.py	Demo §5: columna 10× más rígida
error_support.py	Demo §5: apoyo libre en Z
results/benchmark_results.json	Resultados completos en JSON (se genera al correr <code>benchmark_3d.py</code> ; no versionado, por <code>.gitignore</code>)

Generado con asistencia de OpenCode (IA). Fecha: agosto 2026.